Trust-minimized Bridging Between Federated Byzantine Agreement Blockchains

CHRIS FROST, GIULIANO LOSA, and DAVID MAZIÈRES
Stellar Development Foundation, USA

Federated Byzantine Agreement (FBA) is an open-membership consensus model in which nodes determine quorum composition in a decentralized way. Specifically, each node chooses sets of nodes known as *quorum slices*, and a quorum must contain at least one slice from each of its members. In this paper, we show that the notion of quorum slice can be extended across instances of an FBA protocol, providing a means for nodes in one FBA system to agree on a recent state of another. This capability makes it possible to move assets between FBA blockchains without entrusting the custody of escrowed assets to any external third parties.

1 INTRODUCTION

Blockchain bridging allows assets to be used across multiple blockchains. Typically, an asset is locked up on one chain in exchange for releasing or minting a proxy asset on the destination chain. To move in the other direction, the original, locked asset is released in response to destroying or returning the proxy asset. The key challenge for bridging is allowing logic on one blockchain to verify that something has happened on the other blockchain so as to determine when to release or mint assets.

The simplest bridges employ trusted third parties or committees to release locked assets as appropriate. Unfortunately, such architectures can suffer the theft of locked funds even when the underlying blockchain consensus remains secure [2, 3], rendering the bridge insolvent. The best practice is therefore to employ *trust-minimized* bridging, in which one blockchain directly verifies the consensus output of another blockchain.

Proof-of-work (PoW) blockchains can achieve trust-minimized bridging if each blockchain verifies the cumulative work behind the other's block headers. This verification can be made practical through techniques such as Non-interactive Proofs of Proof of Work [7]. Unfortunately, PoW lacks asymptotic security against a computationally powerful adversary, which particularly undermines blockchains other than Bitcoin [4]. As such, PoW has fallen out of favor for new blockchains.

Proof-of-stake (PoS) blockchains can achieve trust-minimized bridging by ingesting enough of each other's state to track valid signers and verify new block headers. Cosmos IBC [6] demonstrates the approach. Unfortunately, PoS is complicated by long-range attacks, in which validators who have withdrawn their stake can sign alternate histories without risking slashing. Hence, while consensus protocols typically remain live under partial synchrony, IBC with PoS requires human intervention following a sufficiently long network outage.

A third approach to blockchain consensus is Federated Byzantine Agreement (FBA) [9], introduced by the Stellar [8] blockchain. Whereas PoW and PoS use cryptocurrency rewards to incentivize potentially anonymous validators to behave correctly, FBA leverages the real-world reputation of validator operators for security. Operators configure FBA validators to guarantee agreement with other validators based on the need to interoperate with other organizations. Security depends on the so-called *Internet hypothesis*—roughly, the assumption that any two validators one cares about will have some common transitive dependency that prevents them from diverging.

This paper presents a technique for trust-minimized bridging between FBA blockchains. Unlike prior settings, FBA bridging can enjoy both asymptotic security and liveness under partial synchrony. We assume an IBC-like protocol in which validators embed light clients for foreign blockchains. However, unlike PoS, each validator individually configures its light client to ensure agreement

Draft of October 24, 2026.

with specific sets of validators of the foreign blockchain. This "quorum in the eye of the beholder" model naturally extends the FBA trust model across networks.

2 MODEL

This section describes the blockchain model, the federated Byzantine agreement (FBA) model of consensus, and the model for FBA bridging.

2.1 Blockchain model

A blockchain is a replicated state machine. A set of replicas use a consensus protocol to agree on a log, then each deterministically apply the log to update their copy of the state, typically a ledger. Each log entry is known as a *block*, as it generally contains a set of transactions that atomically all commit together when the block is appended to the log. What distinguishes a blockchain from an ordinary replicated log is that consensus takes place on a *block header*, which contains a collision-resistant hash of both the current block and the previous header (except for the first block). Hence, agreement on a block header implies agreement on both the current block and all previous blocks.

Blockchains with smart contracts typically support output *events*. One can implement a bridge by processing events on a foreign blockchain: for every event indicating that an asset was locked or destroyed by a bridge contract on the foreign blockchain, the local bridge contract can release a corresponding local asset. We assume that events emitted on a foreign blockchain can be verified in a local smart contract by hashing given the remote block header and a small amount of auxiliary data such as a Merkle path. (If all bridge events do not fit in calldata, one smart contract invocation can at least verify a range of events.) The key functionality we need for trust-minimized bridging is for smart contracts on one blockchain to be able to verify block headers from a foreign blockchain.

2.2 FBA consensus model

In traditional Byzantine agreement systems, a quorum is a set of validators that is sufficient to make a decision. For example, a protocol for n validators may stipulate that unanimous votes from more than two thirds of the n validators trigger a decision. In this case, every set of validators containing more than two thirds of the validators is a quorum.

Unlike traditional Byzantine agreement, FBA systems determine quorum composition in a decentralized manner. Each validator v of an FBA consensus protocol chooses a set of *quorum slices*, where v only recognizes a set Q as a *quorum* when (a) Q contains at least one slice of v and (b) for every validator v' in Q, Q contains at least one slice of v'. One way to understand this definition is to interpret slices as specifying agreement requirements: a quorum is a set that satisfies the agreement requirements of all its members, and a validator recognizes a quorum when that quorum satisfies the validator's own requirements.

As an example, an FBA system with n nodes can mimic a centralized Byzantine agreement protocol if all validators accept any $\lfloor 2n/3 \rfloor + 1$ nodes as a quorum slice. Alternatively, a validator that wants to agree with 3 specific organizations, each of which runs 3 validators, can choose its quorum slices to include 2/3 of the validators of each organization. Larger quorum slices increase safety at the cost of decreased liveness; at the extreme, choosing a single giant quorum slice containing every validator in the world guarantees agreement with everyone while blocking progress in the face of a single unavailable validator.

Every message broadcast by a validator is digitally signed and specifies the sender's quorum slices (e.g., as a collision-resistant hash). Since validators may choose a large number of quorum slices, protocols should encode slices in a more restricted but compact way. For example, Stellar uses "quorum sets", which specify that any k-of-n items constitute a quorum slice, where each item is either a validator public key or, recursively, another quorum set. Technically, since slices are

specified in messages, the notion of *quorum* applies only to a set of messages—e.g., a quorum of votes for a particular outcome. However, since validators rarely change quorum slices, we often abuse the term to refer to message senders, as in "a quorum voted for the same outcome."

2.3 FBA bridging model

For each foreign blockchain bridged to the local blockchain, we assume that every local validator includes a light client of the foreign blockchain. Every light client has a set of quorum slices for the foreign blockchain. That means each validator operator must choose one set of quorum slices for the local blockchain and one set of quorum slices for each foreign blockchain.

The light client maintains a sequence of *imported* block headers from the foreign blockchain—where imported roughly means that a local quorum believes the header is valid for the foreign blockchain. Whenever the local blockchain is assembling a block to propose to consensus, it asks the light clients for each foreign blockchain to supply the most recent imported block header. A local block that has been decided through consensus references block headers of foreign blockchains and cannot be externalized at a node until the node believes all those foreign references are valid.

To enable bridging, we extend the runtime API to allow smart contracts to ask two questions of light clients:

(1) Is c_i a valid header for foreign blockchain C?
(2) Is c_i the parent of c_{i+1} on foreign blockchain C?

The second question may seem trivial—after all, shouldn't c_{i+1} include a collision-resistant hash of c_i? The problem is that blockchains are not perfect; occasionally they have suffered consensus failures, non-determinism bugs, or logic errors and been unable to recover without rewriting previously externalized history, an occurrence known as a *reorg*. For example, divergent database lock limits once famously led Bitcoin to discard 53 externalized blocks; Bitcoin reexecuted most of the discarded transactions in the reorganized history, but one double-spend had to be rejected [1].

Once a local smart contract has seen a foreign block header c and processed the corresponding events, the effects cannot be locally rolled back even if c is discarded in a reorg of the foreign blockchain. Since trust-minimized bridge contracts often cannot be upgraded, recovering from a foreign reorg requires a way to splice locally externalized, remotely discarded block headers into a history that merges with the reorganized foreign blockchain. This can be done by hard-coding a special *splice* rule into light clients that overrides the normal hash check to consider one specific block to have a parent that differs from the hash value in the block. Inserting such a rule is akin to a protocol upgrade of the local blockchain, which in FBA systems goes through local consensus.

Blockchain reorgs *should* never happen. Unfortunately, historically they have, so we need to anticipate that node operators will need to correct them. To be clear, splicing cannot recover lost funds in the event that a reorg reversed deposits into a foreign bridge contract after the corresponding local tokens were released. Blockchain communities need to repair the damage from reorgs as responsibly and fairly as possible—e.g., possibly finding a way to recapitalize a bridge that has lost funds. The point of splice rules is not that they magically mitigate reorg damage. Rather, the point is to limit the scope of whatever extreme recovery measures people take, so as to ensure that a foreign blockchain reorg does not affect smart contracts that don't care about the foreign blockchain or its assets.

Bridges to foreign blockchains cannot affect the safety of the local blockchain. They should also minimally affect liveness. Unfortunately, there are specific situations in which you actually want a bridge to prevent the local blockchain from producing new blocks, namely when there's a fork or failure of the foreign blockchain and local validators genuinely disagree over the correct state of the foreign chain. That said, three ingredients are all required for a bridge to prevent new blocks:

there must be a foreign validator failure or misconfiguration, there must be a local validator failure, and there must be divergence in the configuration of the local blockchain's light clients. Section 4 discusses some subtleties around the liveness of otherwise non-faulty nodes that have minority light client configurations, but liveness can be protected through norms ensuring identical or highly similar light client configurations at all local validators.

3 PROTOCOL BACKGROUND

Most consensus protocols can be decomposed into a series of *votes*. For optimization reasons, the votes may not be explicit in the protocol messages—for instance, a single message may conceptually cast votes for multiple statements, or, because FBA voting requires two phases, a single message may implicitly piggyback the first phase of one vote onto the second phase of another vote. Nonetheless, to understand consensus protocols, it often helps to think of them in terms of an equivalent conceptual protocol in which all votes are explicit.

The purpose of a vote is to decide among mutually exclusive outcomes. Each non-faulty node *votes* for at most one candidate outcome as input, and may subsequently see a decision outcome as output. When a vote is successful and certain failure assumptions hold (e.g., an n-node Byzantine fault tolerant protocol typically assumes fewer than $n/3$ nodes have failed), all non-faulty nodes see the same decision outcome. Unfortunately, voting can also get permanently stuck if nodes vote for different outcomes, such that no single outcome is supported by a quorum. Worse, with node failure and unknown network delay bounds, it can be impossible to determine whether a vote is permanently stuck or will eventually succeed: a missing vote message could be delayed by the network and eventually complete a quorum, or might never have been sent if the sender failed.

Under the appropriate failure assumptions, voting enjoys the following properties:

- **Non-triviality.** If a non-faulty node decides an outcome, at least one node voted for that outcome.
- **Validity.** If a non-faulty node decides a value, the value is semantically valid at that node.
- **Consistency.** All non-faulty nodes that decide an outcome decide the same outcome.
- **Unanimity.** If all non-faulty nodes unanimously vote for the same outcome, the vote will reach a decision and not get stuck.
- **Totality.** If a vote produces a decision at one non-faulty node and the value is semantically valid at all non-faulty nodes, the vote will produce a decision at all non-faulty nodes.

Non-triviality and validity can easily be enforced at individual nodes. Consistency requires *quorum intersection*—the property that any two quorums share at least one non-faulty node. Unanimity holds under *quorum availability*—the property that there exists at least one entirely non-faulty quorum. In a centralized Byzantine agreement setting, totality can be achieved by broadcasting *quorum certificates*—vote messages digitally signed by a quorum. In the FBA model, different nodes accept different quorums, so there is no notion of a quorum certificate acceptable to all nodes. Achieving totality in the FBA setting requires a two-round federated voting protocol.

3.1 Federated voting

In the FBA model, well-behaved nodes that faithfully obey the protocol can still fail if their quorum slices are misconfigured and allow disjoint quorums. In practice, strong norms [5] prevent this from happening to any validators that matter. Someone could deliberately spin up a set of new, misconfigured validators to create a fork, but no one who follows any of the existing validators would accept any blocks on the branch of the new validators. Given the possibility of disagreement over which well-behaved nodes have faulty configurations, properties such as validity and consistency become properties of *sets of nodes* in FBA, not properties of the whole system.

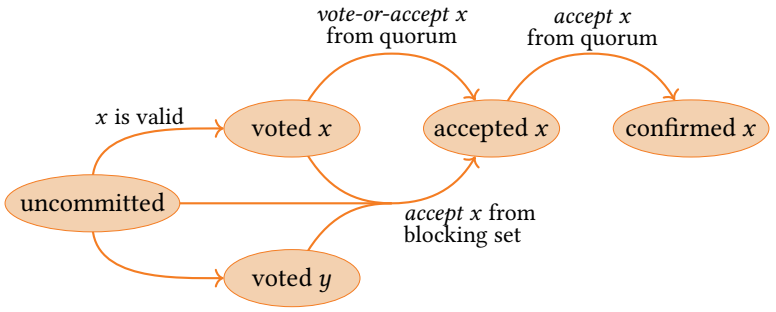


Fig. 1. Stages of federated voting

Federated voting guarantees validity and consistency among *intertwined* sets of nodes, and guarantees unanimity and totality among *intact* sets of nodes. We define these terms precisely in Section 5.2. Informally, nodes are intertwined when any two of their quorums share a well-behaved node, which is essentially a relativized notion of quorum intersection. An intact set is a quorum of well-behaved nodes that remains intertwined even if all non-intact nodes fail. The existence of an intact set is analogous to but stronger than quorum availability; it accounts for the fact that well-behaved nodes can still get stuck in incompatible states if they allow disjoint quorums.

Unfortunately, no single-round voting protocol achieves totality in an FBA setting, because nodes may not even know of the existence of certain other non-faulty nodes, much less what those nodes will accept as a quorum. Moreover, because nodes can adjust their quorum slices unilaterally at any time, even a set of votes that appears to be a valid quorum certificate for another node may no longer be. On the other hand, a node v always knows its own quorum slices, and so knows when enough nodes disagree with it that v can't be in an intact set that decides some value y. At a high level, then, federated voting uses a first quorum to establish that if v is intact, then it can vote for some value x without getting stuck. A second quorum then ensures that intertwined nodes agree on x even when they are not intact.

Consider a set of nodes S that includes at least one node from every one of node v's quorum slices. If every node in S is faulty, v cannot be a member of an intact quorum, since every one of v's quorums will contain at least one node from S and all nodes in S are faulty. We say that a set such as S is *v-blocking* if S intersects every quorum slice of v, meaning that v cannot be intact unless at least one member of S is non-faulty.

Figure 1 depicts the federated voting protocol at each node v. First, v *votes* for one among several mutually exclusive outcomes (abstractly x or y in the diagram) by broadcasting a digitally signed vote message including a hash of v's quorum slices. If v is part of a quorum that unanimously votes for some value x, then v *accepts* x and broadcasts a new message declaring it has accepted x, once again including a hash of its quorum slices. Whether or not v voted for x, if a v-blocking set has unanimously accepted x, v also accepts x and broadcasts this fact. The fact that a v-blocking set has accepted x means that if v is in an intact set, it could never be in a quorum for a value other than x. Finally, if v is in a quorum in which every node has accepted x, v *confirms* x and locally outputs x as the decision of the vote.

3.2 Externalize messages

Though the specifics of the FBA consensus protocol do not affect bridge design, once a node chooses a block through consensus, it generally digitally signs an *externalize* message stating that it has committed the block. Externalize messages are not strictly required for normal operation, but help

nodes that have fallen behind catch up. An externalize message signed by node v for block b means that not only did v reach the accept phase of whatever federated vote was required to choose b, but v also verified that every other member of a quorum accepted b. Later on, it could be that some members of that quorum left the system and no longer exist, so it is handy not to have to worry about them. Where ordinarily you need to check a quorum of messages, you only need to check a quorum *slice* of externalize messages.

Light clients primarily ingest externalize messages from a foreign blockchain. These are particularly useful to bridges for two reasons. First, it is simpler to verify foreign quorum slices than foreign quorums. Second, because externalize messages are not required by the FBA protocol, they provide an opportunity to piggyback additional assertions in a digitally signed message. In particular, we can modify the semantics of externalize messages to include additional assertions about foreign blockchains.

4 INFORMAL PROTOCOL

The job of a light client for blockchain C, embedded in validators of blockchain B, is to determine which block headers are *valid* on C and to maintain a growing sequence of *imported* block headers from C. Each imported block header in the sequence is the parent of the next block header, which (barring a reorg of C) means that the child contains a collision-resistant hash of the parent.

To recover from a reorg of C, B's light clients may hard-code a few pairs of the form $\langle c_P, c\rangle$ in a *splice table*. This states that c_P is the parent of c, even if c's parent pointer does not match a hash of c_P. Moreover, it makes c_P inherently valid and imported, without regard to messages from other nodes. The rest of this section explains the normal case of how blockchain B imports and validates block headers from blockchain C in the absence of splices.

If v is a B-validator (i.e., a validator for blockchain B), a block header c of blockchain C is *valid at* v if v's light client has seen externalize messages for c from one of its quorum slices for C. We assume that light clients in B-validators monitor and multicast C's externalize messages among themselves. Hence, if a non-faulty light client in a B-validator sees an externalize message from a C-validator, eventually all light clients in non-faulty B-validators will see the same message. Nodes disseminate externalize messages regardless of consistency. If a faulty C-validator equivocates, signs two contradictory externalize statements, and sends them to two different non-faulty B-validators, eventually both B-validators will see both externalize messages. This means multiple foreign block headers with the same parent can both be valid at the same B-validator.

Light clients in B-validators *import* block headers from C by conducting a federated vote on statements of the form:

$$\textsc{import}\ C\ c_P\ c.$$

A B-validator v can only vote for such a statement if c is valid at v and c_P is c's parent. In addition, v can only vote for one child c for a given parent c_P, and can only accept one value of c for a given c_P. However, following the rules of federated voting, v can accept an IMPORT statement without voting for it (and hence without yet believing it valid) if a v-blocking set accepts the statement. When v confirms $\textsc{import}\ C\ c_P\ c$, then c is considered imported by v.

In the absence of bridges, the consensus values decided by FBA blockchains are actions of the form, "Append block header b to blockchain B." With bridges, the consensus values are actions conditioned on the validity of foreign block headers, which can be read as, "Append block header b (which references foreign block header c) to blockchain B *if c is a valid block header for blockchain C*." Consensus leaders should set c to the latest imported block header from C in block proposals. Block validators should accept any imported c that is no older than the c in the previous block.

To make externalize messages actionable, we modify their semantics to remove any caveats about foreign blockchains. In other words, after consensus at B-validator v decides that b is valid if c is valid, v cannot sign an externalize message for b or expose b to client-facing services until c is valid at v.

A few informal observations motivate this design. Suppose blockchain B has an intact quorum in which every validator's light client has the exact same set of quorum slices for blockchain C. Then no failure of C-validators can impact B's liveness, because all intact B-validators will agree on which blocks of C are valid, so totality of the IMPORT vote ensures eventual agreement on any valid c.

Suppose, alternatively, that B never suffers a node failure, but that its light-client configurations are an incoherent mess and that blockchain C forks. Once again, B will keep producing blocks, as any foreign block header c proposed to FBA consensus must have received a quorum of IMPORT votes. There is one subtlety, however: a well-behaved B-validator v could stop externalizing blocks if a quorum of validators with different light client configurations imports a block that v considers invalid. Whereas v would be intact without the bridge, its subjectively "faulty" (in the eyes of a quorum) choice of quorum slices on C can cost v liveness.

A bridge can also prevent all B-validators from externalizing new blocks in the event that blockchain C forks, there is genuine disagreement between B-validators as to which branch of the fork constitutes the "real" blockchain C, and some B-validators also fail. Suppose a "c-valid" faction of B-validators believes c is the next block header for C, while a "c-invalid" faction believes that either $c' \neq c$ is the next valid block header, or there is no later valid header. If neither faction contains a quorum, B will continue to be live, as no further IMPORT votes can succeed for C. On the other hand, if the c-valid faction contains a quorum, it can continue to commit new blocks to B referencing c and c's descendants, while the c-invalid faction eschews these new blocks. If, at some point, some B-validators in the c-valid faction fail, B can lose liveness, even though, in the absence of a bridge, there would be an intact quorum spanning the two factions.

An analogous liveness issue arises with protocol updates: a quorum of B can vote to upgrade the software to a version of the protocol that is not supported by all validators. If some of the nodes that voted to upgrade subsequently fail, the network can lose liveness even when there would be a quorum at the old protocol version. The solution is to upgrade the remaining nodes to support the new protocol. Similarly, in the bridge scenario, a solution to regain liveness is to synchronize foreign quorum slices across light clients. One of the benefits of FBA consensus is that nodes can change their quorum slices at any time, with no need to agree on the reconfiguration through consensus.

5 FORMALIZATION

In this section, we formalize the proposed bridge architecture and show its exact security properties.

5.1 Network model

We consider a message-passing distributed system consisting of a finite set of nodes N. Each node in N faithfully runs the protocols that it is assigned, except for an unknown, fixed subset F of faulty nodes, which may deviate arbitrarily from any protocol they are assigned. We say that the other nodes are well-behaved and we define the set of well-behaved nodes $W = N \setminus F$.

We assume that each node may broadcast messages and that there is a constant Δ, unknown to the nodes, such that every message broadcast by a well-behaved node is received by all well-behaved nodes within Δ time. This is a simpler variant of the traditional eventual-synchrony assumption, where the bound may be known but it holds only after an unknown time called the global stabilization time (GST). We assume that each node has a unique public/private key pair

allowing it to authenticate the messages it sends by signing them, and that faulty nodes cannot forge signatures.

We further assume that each node in the system may be a validator of at most one blockchain among a finite set of blockchains $\mathcal{B}$. We write V_B for the set of validators of a blockchain $B \in \mathcal{B}$ and assume that each validator $v \in V_B$ has a non-empty set $\mathcal{S}_B(v)$ of non-empty subsets of V_B called quorum slices. This determines an FBA configuration $\Sigma_B = (V_B, \mathcal{S}_B)$, as described in the next section.

By convention, when a notion is relative to a blockchain B, we may use the prefix "B-" to indicate this. For example, we may say that a node is a B-validator to mean that it is a validator of blockchain B.

5.2 FBA configurations

An FBA configuration Σ is a pair $(V, \mathcal{S})$ where $V \subseteq N$ is a set of validator nodes, and, for every validator $v \in V$, $\mathcal{S}(v)$ is a non-empty set of sets of validators called v's quorum slices. Now consider an FBA configuration $(V, \mathcal{S})$.

We say that a set Q of validators is a quorum when every member of Q has a quorum slice included in Q. Moreover, Q is a quorum of a validator v when it additionally includes a slice of v. As defined in Section 3.1, a set of validators V' blocks a validator v, or is v-blocking, if every slice of v intersects V'.

Given a set of validators $V' \subseteq V$, the projection $\Sigma \restriction V'$ removes all validators not in V', truncating each slice to V'. We define it as $\Sigma \restriction V' = (V', \mathcal{S}')$ where, for every validator $v \in V'$, $\mathcal{S}'(v) = \{S \cap V' \mid S \in \mathcal{S}(v)\}$. Conversely, given a set of nodes X, deleting X from Σ yields $\Sigma \ominus X = \Sigma \restriction (V \setminus X)$, as in the Stellar whitepaper [9].

Note that both quorum slices and quorums may be empty after a projection, and so our definitions above allow empty quorum slices and empty quorums. However, validators in a real network should not choose empty quorum slices.

We say that a set of validators V' has quorum intersection despite another set X of validators when, in $\Sigma \ominus X$, every two quorums of members of V' intersect. Moreover, V' has quorum availability despite X when every member of V' has a quorum disjoint from X (in the original configuration Σ).

We say that a set of validators V' is intertwined when V' is non-empty and well-behaved, and V' has quorum intersection despite the set of faulty validators F. Two validators v and v' are intertwined when $\{v, v'\}$ is intertwined. We say that a set of validators I is intact when I is non-empty and well-behaved, I is a quorum, and I has quorum intersection despite its complement. We say that a set is intact in blockchain B, or B-intact, when it is intact in Σ_B, and similarly for intertwined. Note that intact sets are sets that are self-sufficient in the sense that they have quorum intersection and availability despite their complement.

It is easy to see that every intact set is intertwined. Although a bit technical, we can also prove that the union of two intersecting intact sets is an intact set; since the notion of agreeing on something is inherently transitive, this makes sense if members of intact sets are to agree with each other. Finally, intact sets satisfy the cascade theorem:

Theorem 5.1 (Cascade). *Let I be an intact set, Q a quorum that intersects I, and S a set that includes Q. If $I \not\subseteq S$, then some member of $I \setminus S$ is blocked by $I \cap S$.*

Thus, starting from $S = Q$ and repeatedly adding to S the members of I that $I \cap S$ blocks eventually covers I. This guarantees that once the members of I in a quorum agree on something, the agreement propagates to all of I. Appendix A gives the proof.

5.3 Block Headers and Blocks

A block header b is a data structure associated with a block, denoted $\mathsf{block}(b)$, and a parent, denoted $\mathsf{parent}(b)$. The parent is either another block header or the special sentinel value $\perp$ for a block with no parent. Block headers generally specify their block and parent using collision-resistant cryptographic hash values.

In a bridged blockchain, a block comprises a transaction set and foreign block headers. The transaction set is the same as in a non-bridged blockchain, typically a set of transactions, a state hash, and a set of output events. The foreign block headers correspond to the latest known state of each foreign blockchain as of when the block was assembled. We write $\mathsf{fh}(b)$ for the set of foreign block headers appearing in the block associated with header b.

We say that a block header b' is an ancestor of a block header b and, equivalently, that b is a descendant of b', when $b' = b$ or b' is reachable from b by following the parent relation. We assume that the parent relation is acyclic and well founded (meaning that every block has a finite chain of ancestors). We say that two block headers b and b' are compatible when one is an ancestor of the other, and incompatible otherwise.

Each blockchain B has a fixed, trusted genesis header g_B, known to all nodes, with $\mathsf{parent}(g_B) = \perp$. Its block specifies the initial application state and has no foreign headers: $\mathsf{fh}(g_B) = \emptyset$. When a block has no header for a foreign blockchain C, we interpret its C-header as $\perp$. By convention, $\perp$ is an ancestor of itself and of every foreign header in every blockchain's view. It is valid and imported by default.

We abstract away block dissemination: we assume that a validator that obtains a block header also obtains its ancestors and, for headers of its own blockchain, their blocks.

To recover from reorgs of a foreign blockchain C, a blockchain B may override C's parent relation using a splice table T_B^C, a finite partial map from C-headers to C-headers used by B's light clients for C. The parent of a C-header c in B's view, written $\mathsf{parent}_B(c)$, is $T_B^C(c)$ if T_B^C maps c, and $\mathsf{parent}(c)$ otherwise. We define ancestors, descendants, and compatibility of C-headers in B's view as above, but following parent_B instead of parent, and we assume that parent_B is acyclic and well founded. We call the ancestors in B's view of the headers in the image of T_B^C spliced. They include headers that C abandoned and synthetic headers that C never externalized, created by B's operators along with their blocks. For example, if C abandons a header y that B has referenced, T_B^C can map a later C-header r to a synthetic header y' with parent y, so that B can follow C again from r on (Figure 2); then y', y, and their ancestors in B's view are spliced. Section 5.6 explains how operators build splices.

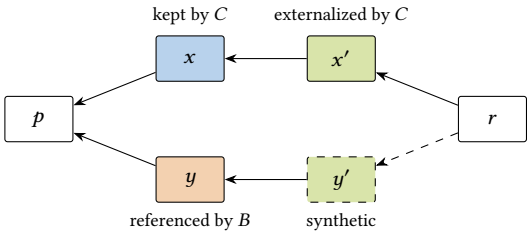


Fig. 2. Repairing a reorg of C with the splice table entry $T_B^C(r) = y'$. All headers shown are C-headers. Solid arrows point from a header to its parent; the dashed arrow points from r to its parent in B's view.

We take T_B^C as fixed and known to all B-validators. In practice, operators add entries through coordinated protocol upgrades, ensuring compatibility with existing import votes and committed

blocks, with no affected imports or block proposals in flight. We leave upgrade coordination outside our model.

We say that a C-header c is above B's splices for C when no splice applies to a strict descendant of c, either under C's parent relation or in B's view. For example, in Figure 2, r and its descendants are above B's splices, but the other headers shown are not.

5.4 The Bridged FBA Blockchain Interface and its Guarantees

An FBA blockchain B is a distributed protocol run by the set of B-validators V_B and bridged to a fixed set $\mathsf{foreign}(B) \subseteq \mathcal{B} \setminus \{B\}$ of blockchains called B's foreign blockchains. Figure 3 shows the components of a B-validator. The application is typically a replicated ledger supporting smart contracts, and it is what users of the blockchain interact with. The consensus layer agrees on B-headers. The block manager supplies consensus with new headers to propose and tells it which headers it may vote for (their admissibility). It passes committed headers to the application and externalizes them once their foreign headers are valid. For each foreign blockchain C, the light client for C determines which C-headers are valid from C's externalize messages, and imports C-headers by federated voting among B-validators (Section 4).

We specify B as a black box made of consensus, the block manager, and the light clients. Its interface consists of the procedures through which it interacts with the application (Section 5.4.1), the externalize messages that its validators broadcast, and the externalize messages from foreign blockchains from which its light clients determine which foreign headers are valid (Section 5.4.2). Section 5.7 describes how the components implement this interface.

5.4.1 Application interface. At each well-behaved B-validator, the application starts with the trusted genesis header g_B committed and externalized. B interacts with the application through four procedures:

- $\mathsf{commit}(b)$ notifies the application that header b, and by convention each of its ancestors, is committed. A committed header is final and new blocks may build on it, but the application must not act on it, e.g., by reporting the effects of its transactions to users, until it is externalized.
- $\mathsf{externalize}(b)$ notifies the application that the committed header b, and by convention each of its ancestors, is externalized. Externalizing a header b also implies broadcasting the externalize message EXTERNALIZE B b, and a well-behaved validator sends no externalize message for a header that it has not externalized.
- $\mathsf{get_txset}(b, H)$ requests a transaction set for a new block with parent b and foreign headers H, where H contains exactly one C-header for each $C \in \mathsf{foreign}(B)$.
- $\mathsf{is_txset_valid}(b)$ asks the application whether the transaction set in b is valid with respect to b's parent and foreign headers.

Since the blockchain may commit a block well before externalizing it, the application must validate and build on blocks whose parents are committed but not yet externalized.

To state our assumptions about this interface, we first define conditional B-validity. Commitment requires the transaction set and ancestry of a block to be valid, while externalization also requires its foreign headers to be valid. Conditional B-validity expresses the former requirement.

The genesis header g_B is conditionally B-valid. Any other B-header b is conditionally B-valid when $\mathsf{is_txset_valid}(b)$ holds, $\mathsf{parent}(b)$ is conditionally B-valid, and, for each foreign blockchain C, the C-header of b is a descendant in B's view of the C-header of $\mathsf{parent}(b)$.

We assume:

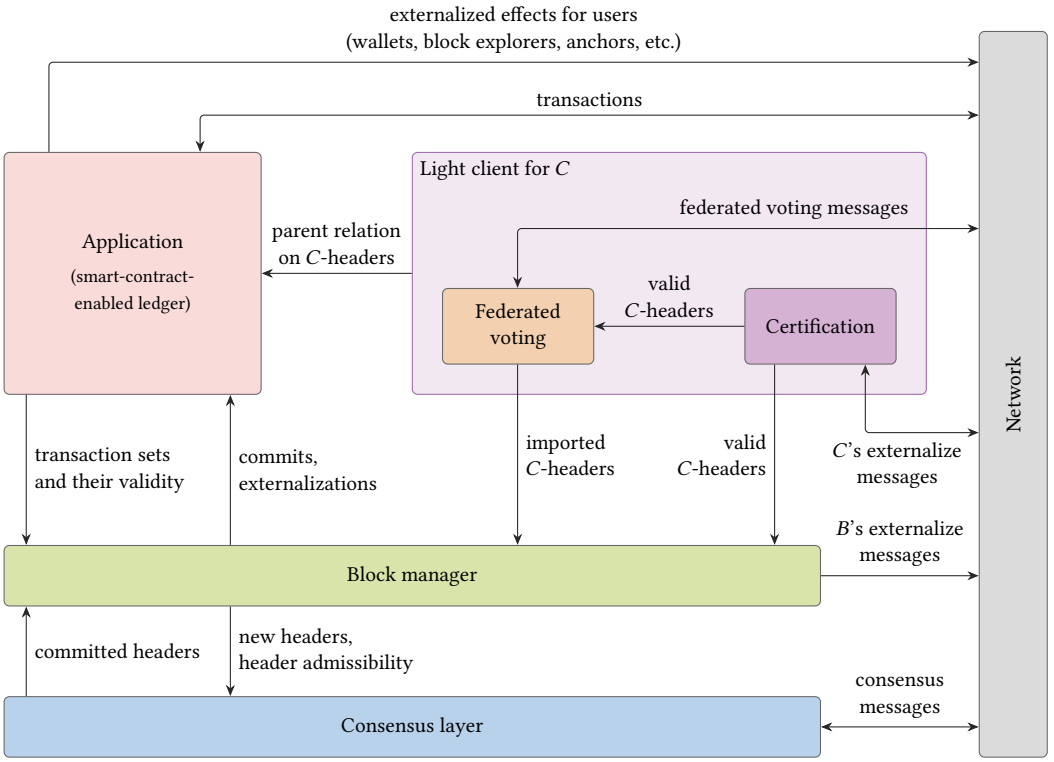


Fig. 3. Component stack at each B-validator, for a single foreign blockchain C; with several foreign blockchains, each has its own light client. Arrows show the flow of information; arrows to and from the network show messages sent and received.

- The result of is_txset_valid(b) depends only on b and its ancestors. At every well-behaved validator, checking conditional B-validity of a header, including any unchecked ancestors, takes at most a fixed time Δ_{val}.
- If b_P is conditionally B-valid and each C-header in H descends in B's view from the C-header of b_P, then get_txset(b_P, H) eventually returns a transaction set T such that the header b of the block with parent b_P, foreign headers H, and transaction set T satisfies is_txset_valid(b).

5.4.2 Light clients. For each foreign blockchain C, each B-validator v has a non-empty set $\mathcal{S}_B^C(v)$ of quorum slices for C, called C-slices, used by its light client for C. Each C-slice is a non-empty set of C-validators. This determines B's light-client configuration $\Sigma_B^C = (V_B, \mathcal{S}_B^C)$ for C.

Light clients use externalize messages from their foreign blockchain to determine which foreign headers are valid. We call a set of externalize messages from a foreign blockchain C a v-certificate for a C-header c when it contains a signed EXTERNALIZE C c message from every member of some C-slice of v. We say that a C-header c is valid at a validator v when $c = g_C$, c is spliced, or v's light client has obtained a v-certificate for c. Since externalizing a header externalizes its ancestors, a v-certificate for c is also a v-certificate for every ancestor of c. Spliced headers are valid by default because no certificate for a later C-header covers them, as for y and y' in the example of Section 5.3.

The set of C-headers valid at v only grows. Valid headers need not be compatible: for example, if C's validators equivocate, conflicting C-headers may both be valid at v.

The light client for C also answers the application's question is_parent(c_P, c), which holds when $c_P = \mathsf{parent}_B(c)$. This lets smart contracts ask whether a C-header c is valid: a contract executing in a block with C-header c', determined by H in get_txset(b_P, H) or by $\mathsf{fh}(b)$, considers c valid when c is an ancestor of c' in B's view, which it checks using is_parent. If a header is valid at v, so are its ancestors in B's view. By the Validity property below, every foreign header a contract considers valid is therefore valid at v once v externalizes the block.

5.4.3 Correctness Properties. To state the correctness properties, we first define B-validity, which externalization requires, and total foreign-header validity, which our liveness guarantees assume.

We say that a block header b is B-valid at a validator v when it is conditionally B-valid and each foreign header of b and of its ancestors is valid at v. In comparison, conditional B-validity drops the requirement that foreign headers be valid at v.

We say that foreign-header validity is total in a set V' of well-behaved B-validators when, for every foreign blockchain C of B, every C-header valid at one member eventually becomes valid at every member. In other words, foreign-header validity must eventually propagate, with no fixed bound on the delay.

A blockchain B must satisfy the following properties:

- **Validity.** If a well-behaved B-validator v commits a block b, then b is conditionally B-valid, and if v externalizes b, then b is B-valid at v.
- **Consistency.** A well-behaved B-validator never commits incompatible blocks, and two intertwined B-validators never commit incompatible blocks.
- **Totality.** For every B-intact set I, if a member of I externalizes a B-block b, and each foreign header of b and of its ancestors eventually becomes valid at a member v of I, then v eventually externalizes b.
- **Liveness.** For every B-intact set I in which foreign-header validity is total, and every $v \in I$, v externalizes infinitely many B-blocks whose transaction set was returned at a member of I by get_txset(b_P, H), where b_P and H are the block's parent and foreign headers.
- **Foreign liveness.** For I as in Liveness, let b be a B-block externalized by a member of I. Let c_b be the C-header of b, and let c be a C-header descending from c_b in B's view. Suppose c becomes valid at a member of I, and no C-header that descends from c_b and is incompatible with c, both in B's view, is ever valid at a member of I. Then every member of I eventually externalizes a B-block with a C-header that is a descendant of c in B's view.

We now make some observations about these properties. Since a validator externalizes only blocks that it has committed, Consistency also holds for externalized blocks. Validity implies **Foreign validity**: if a well-behaved B-validator v externalizes a B-block with C-header c, then c is valid at v. Moreover, since C-headers descend in B's view along committed chains, Consistency implies that the C-headers of blocks committed by a well-behaved B-validator, or by two intertwined B-validators, are compatible in B's view.

B-validators may use different C-slices, so a C-header valid at one validator need not become valid at the others. A committed block referencing that header could therefore remain unexternalized at those validators. This is why Liveness and Foreign liveness assume total foreign-header validity. Foreign-header validity is total in V' if its members use the same C-slices for each foreign blockchain C and relay externalize messages: a certificate for one member then suffices for all. Section 5.5 gives another sufficient condition allowing different foreign slices.

Foreign liveness is stated relative to an externalized block because both sides of a repaired reorg remain valid (x and y in Figure 2); only conflicts above B's current C-header can prevent progress.

A validator broadcasts EXTERNALIZE B b when it externalizes b, not when it commits b, so that, by Foreign validity, the message also vouches for the foreign headers of b. This keeps a fail-safe

for bridges in the other direction: if C commits a C-header c whose block contains a B-header that is not spliced in C's view and that no member of a B-intact set I_B ever externalizes, e.g., because C's light clients are misconfigured, then a well-behaved C-validator whose B-slices all intersect I_B never externalizes c. If c is also not spliced in B's view, it is never valid at a B-validator whose C-slices each contain such a validator.

5.5 Cross-Chain Guarantees

We now consider the guarantees obtained by bridging two blockchains that each satisfy the preceding properties. Consistency requires no additional assumptions on foreign slices or the behavior of foreign validators. We give conditions on local and foreign quorum configurations that ensure cross-chain validity and liveness. For liveness, the crux is to ensure that foreign-header validity is total.

Identical foreign slices give a first sufficient liveness condition:

Theorem 5.2. *Suppose the members of a B-intact set I_B use the same C-slices for each foreign blockchain C and relay externalize messages. Then foreign-header validity is total in I_B, and every member of I_B externalizes infinitely many B-blocks, regardless of the behavior of foreign validators.*

Proof. Genesis and spliced headers are valid at every member by definition. For any other foreign header, a certificate obtained by one member suffices for all and eventually reaches them through relaying. Thus foreign-header validity is total in I_B, and Liveness gives the externalization guarantee. □

For cross-chain validity, we next relate B's C-slices to C's local slices. Consider the FBA configuration $\Sigma_C \cup \Sigma_B^C$, in which each C-validator has its local slices and each B-validator has its C-slices. (Since no C-slice contains a B-validator, adding the B-validators does not change the quorums of C-validators.)

We say that a set V' of B-validators is C-aligned with a set I_C of C-validators when $V' \cup I_C$ is intact in $\Sigma_C \cup \Sigma_B^C$. We say that a B-intact set I_B and a C-intact set I_C are mutually aligned when I_B is C-aligned with I_C and I_C is B-aligned with I_B.

Alignment first guarantees that the C-headers in externalized B-blocks are spliced or were externalized by C:

Theorem 5.3. *Let V' be a set of B-validators that is C-aligned with a set I_C of C-validators. If a member of V' externalizes a B-block containing a C-header c, then c is spliced or a member of I_C externalized c, in which case c is compatible with every C-header externalized by a member of I_C.*

If the members of V' are moreover intertwined in B, the C-headers of the B-blocks that they externalize are also compatible with each other in B's view (Section 5.4.3).

With mutual bridges, whether foreign-header validity is total in I_B depends on externalization in C, which depends in turn on which B-headers are valid at C-validators. The blockchain Totality property breaks this circularity, by induction on the causal order of externalizations:

Theorem 5.4. *Suppose that B and C are each other's only foreign blockchains, and that a B-intact set I_B and a C-intact set I_C are mutually aligned. Then foreign-header validity is total in I_B and in I_C.*

Foreign-header validity is then total, but not necessarily timely (Section 5.8).

Corollary 5.5. *Under the hypotheses of Theorem 5.4, suppose moreover that a member of I_B externalizes a B-block containing a C-header c_0 above B's splices for C. If a member of I_C externalizes a descendant c of c_0, then every member of I_B eventually externalizes a B-block with a C-header that is a descendant of c. The same holds with B and C exchanged.*

Theorem 5.4 and Corollary 5.5 generalize to any number of blockchains with any bridges between them, provided that each blockchain X has an X-intact set I_X that is Y-aligned with I_Y for each foreign blockchain Y of X.

5.6 Repairing Foreign Reorgs with Splices

Without splices, B can never follow C again once C abandons a header that B has referenced. In Figure 2, once B commits a block with C-header y, the C-header of every later B-block must descend from y in B's view (Section 5.4.1), and none of the headers that C externalizes after the reorg does. The splice $T_B^C(r) = y'$ makes r a descendant of y in B's view, so B's C-header can advance from y to r. Since r is above B's splices, Corollary 5.5 then applies with $c_0 = r$ once a member of I_B externalizes a B-block with C-header r.

A splice does not by itself repair the damage of a reorg. If B's application has acted upon events of y, such as deposits into a bridge contract, that C no longer contains, the communities of B and C must agree, informally, on a remediation plan. The splice and any synthetic headers implement B's side of the plan without B itself forking, so that the reorg affects only the contracts that use C's events. In Figure 2, for example, C can externalize x', whose block replays the events of y that x lacks, followed by r, and B's operators can create y', whose block replays the events of x that y lacks. Then the ancestors of r in B's view, $r, y', y, p, \ldots$, carry the same events as its ancestors, $r, x', x, p, \ldots$, each exactly once. If some events of y cannot be replayed on C, for example because they include a double spend, the communities can instead agree on another plan, such as burning the excess tokens on B on a schedule.

Operators distribute the blocks of synthetic headers with the entry and coordinate its installation as described in Section 5.3. Each blockchain that bridges from C decides independently whether to splice: one that referenced x needs no splice to follow C to r.

5.7 Implementation

We now outline how the components of Figure 3 provide the interface and properties of Section 5.4.3. The main constraint is that consensus needs timely admissibility for liveness: a header eligible for voting at one member of an intact set must become eligible at every member within a fixed delay. Foreign-header validity need not satisfy this constraint. Consensus therefore requires foreign headers to be imported through federated voting; externalization additionally requires them to be valid.

At each well-behaved B-validator, consensus starts with the genesis B-header g_B committed, and each light client for C starts with the genesis C-header g_C imported. Since foreign genesis headers are valid by definition, the first new B-block can reference them without waiting for messages from other blockchains.

5.7.1 Light clients. At each B-validator v, the certification component of the light client for C determines which C-headers are valid at v, and relays the externalize messages from C that it obtains to the other B-validators. The federated-voting component runs federated voting (Section 3.1) among B-validators, with their slices in Σ_B, on statements IMPORT C c_P c (Section 4), where $c \neq g_C$ is a C-header and $c_P = \text{parent}_B(c)$ is its parent in B's view ($T_B^C(c)$ if the splice table maps c, and $\text{parent}(c)$ otherwise). We say that two such statements conflict when they have the same c_P and different c. A well-behaved v votes for at most one, and accepts at most one, of any two conflicting statements. It votes for IMPORT C c_P c once c is valid at v, and never before, but it accepts IMPORT C c_P c following the rules of federated voting, whether or not c is valid at v.

We say that a non-genesis header c is imported at v when v confirms IMPORT C c_P c and c_P is an imported C-header at v. Since v accepts at most one child of each C-header, the C-headers imported

at v form a chain starting at g_C in B's view, whose last element is v's latest imported C-header. If C's validators equivocate, the votes of B-validators may split between two children of a C-header, and B may then never import beyond it; Foreign liveness excludes this case by hypothesis.

Lemma 5.6. *Let I be a B-intact set and c a C-header imported at a member of I at time t. Then c is valid at a member of I, and c is imported at every member of I by time $t + |I|\Delta$.*

The lemma holds for g_C by initialization. For other headers, the first part holds because the first member of I to accept an IMPORT statement must have voted for it. The second follows from the cascade theorem (Theorem 5.1), since members of I accept IMPORT statements through blocking sets whether or not the header is valid at them. Appendix A gives the proof.

5.7.2 Consensus. We say that a B-header b is admissible at a validator v when b is conditionally B-valid and each foreign header of b and of its ancestors is imported at v. Once admissible at v, b stays admissible at v. The block manager supplies the consensus layer with new headers to propose and with their admissibility, and the consensus layer of a well-behaved v votes only for headers admissible at v.

We say that admissibility is timely in a set V' of B-validators when there is a fixed bound D such that, if a B-header is admissible at a member of V' at time t, then it is admissible at every member of V' by time $t + D$. We assume that the consensus layer guarantees the following, for every B-intact set I:

- **Validity.** If the consensus layer of a member of I commits a header, then the header is admissible at a member of I.
- **Consistency.** The consensus layer of a well-behaved B-validator never commits incompatible headers, and those of two intertwined B-validators never commit incompatible headers.
- **Totality.** If the consensus layer of a member of I commits a header, then those of all members of I eventually commit it.
- **Liveness.** If admissibility is timely in I, then the consensus layers of members of I commit infinitely many headers proposed by members of I.

These are the usual guarantees of consensus protocols for partial synchrony, adapted to the FBA model [9].[1]

Liveness needs a fixed bound because consensus relies on timeouts, which must eventually exceed the time that members of I take to find each other's proposals admissible; like Δ, D need not be known. Conditional B-validity depends only on the header and its ancestors, and can be checked within Δ_{val}. Together with Lemma 5.6, this implies that admissibility is timely in every B-intact set I, with $D = |I|\Delta + \Delta_{val}$. This bound depends neither on C nor on B's C-slices, whereas foreign-header validity need not be timely in I, even under mutual alignment (Section 5.8).

Requiring only that headers be conditionally B-valid would also be timely, but too weak: a faulty proposer could then get members of I to commit a block whose C-header is never valid at any member of I, and no member of I could externalize that block, or any later block, since later blocks descend from it. Requiring imports through federated voting rules this out, because a header imported at a member of I is valid at a member of I (Lemma 5.6).

[1] SCP's liveness additionally assumes periods without new messages from faulty nodes; otherwise, faulty nodes can perpetually preempt progress. Since this is not inherent to the FBA model, we assume the stronger liveness property stated above. A weaker liveness property, such as SCP's, could also be used with timely admissibility; our Liveness and Foreign liveness guarantees would then inherit its additional assumptions.

5.7.3 Block manager. The block manager of a well-behaved B-validator v follows these rules:

- To propose a header with parent b_P, v obtains a transaction set from get_txset(b_P, H), where H contains v's latest imported C-header for each foreign blockchain C. If b_P is admissible at v, so is the new header: the C-header of b_P is imported at v, and thus an ancestor in B's view of the latest.
- v calls commit(b) once its consensus layer commits b and b is admissible at v, and externalize(b) once, moreover, b is B-valid at v.

Theorem 5.7. *If the consensus layer provides the guarantees above and well-behaved B-validators follow the rules of this section, then B satisfies the properties of Section 5.4.3.*

Validity and Consistency follow from the rules and from the corresponding guarantees of consensus. For the other properties, consider a B-intact set I. Since admissibility is timely in I, consensus keeps committing headers, and, by Lemma 5.6, their foreign headers are valid at a member of I; if foreign-header validity is total in I, these foreign headers thus become valid at every member of I, which can then externalize the headers. Appendix A gives the proof.

5.8 Mutual Bridges and Unbounded Delays

The following example shows that foreign-header validity can be total without being timely, even under mutual alignment. Suppose B and C each have three validators, v_1, v_2, v_3 and v'_1, v'_2, v'_3. On B, v_1 has local slices $\{v_1, v_2\}$ and $\{v_1, v_3\}$ and a single foreign slice $\{v'_1\}$, v_2 has a single local slice $\{v_1, v_2\}$ and a single foreign slice $\{v'_2\}$, and v_3 is faulty; the situation is symmetric on C (Table 1). Then $\{v_1, v_2\}$ and $\{v'_1, v'_2\}$ are intact and mutually aligned. Suppose that messages among v_1, v_3, v'_1, and v'_3 are instantaneous, while messages to or from v_2 or v'_2 take Δ.

Table 1. Configuration of the mutual-bridge example.

Validator	Blockchain	Local slices	Foreign slice
v_1	B	$\{v_1, v_2\}$, $\{v_1, v_3\}$	$\{v'_1\}$
v_2	B	$\{v_1, v_2\}$	$\{v'_2\}$
v_3	B	Faulty	
v'_1	C	$\{v'_1, v'_2\}$, $\{v'_1, v'_3\}$	$\{v_1\}$
v'_2	C	$\{v'_1, v'_2\}$	$\{v_2\}$
v'_3	C	Faulty	

First, v_3 and v'_3 cooperate: v_1 commits with the quorum $\{v_1, v_3\}$ and v'_1 with $\{v'_1, v'_3\}$, alternating every $\delta < \Delta$, each new block containing the other blockchain's latest header (Figure 4). Since v_2 accepts import statements through the blocking set $\{v_1\}$, without waiting for their headers to be valid at v_2 (Lemma 5.6), v_2 commits each B-block within a fixed delay of v_1. To externalize it, however, v_2 must wait for v'_2 to externalize its C-header, which in turn requires v_2 to have externalized the previous B-block. Thus v_2 externalizes at most one B-block every 2Δ while v_1 commits one every 2δ, so v_2's lag, and the number of blocks that v_2 has committed but not externalized, grow with each block. Foreign-header validity is total in $\{v_1, v_2\}$ (Theorem 5.4), but takes ever longer to propagate. The lag is bounded when certificates transfer between validators: if v_2's C-slice were $\{v'_1\}$, for example, v_2 would externalize each block within a fixed delay of committing it.

Now suppose that v_3 and v'_3 go silent, so that $\{v_1, v_2\}$ is the only quorum through which v_1 can commit. B keeps committing blocks, since admissibility remains timely, but it imports no new C-header until v_2 catches up, since importing a C-header requires v_2's vote, which requires the

header to be valid at v_2. This takes time proportional to the length of the first phase and can be arbitrarily long. However, it is consistent with Liveness and Foreign liveness, which only make eventual guarantees.

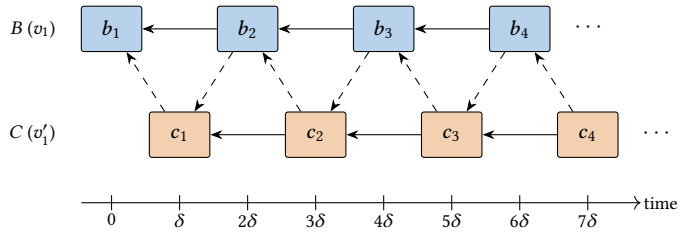


Fig. 4. Blocks committed by v_1 on B and by v_1' on C during the first phase of the mutual-bridge example, placed according to when they are committed. Solid arrows point from a block to its parent; dashed arrows point from a block to the foreign header it contains, which is the other blockchain's latest header.

6 CONCLUSION

The notion of quorum slice from FBA consensus can be extended across different replicated state machines, allowing multiple blockchains to track each other's block headers. We present a technique that embeds light clients for foreign blockchains at every local validator, where each light client is configured with quorum slices of validators of the foreign blockchain. A light client considers a remote block header valid when the header has been unanimously externalized by one of the light client's quorum slices. Before referencing the tip of a foreign blockchain in a block proposed to the local consensus protocol, local validators must vote to import the tip. Before externalizing a local block, validators must ensure that any foreign block headers referenced in the local block are valid.

Our technique exposes foreign block headers through an API allowing smart contracts to verify foreign block headers and their parents. This API is precisely what is required to implement bridges that move assets across blockchains. Unlike prior approaches for proof-of-work and proof-of-stake blockchains, FBA bridging enjoys both asymptotic security (in the security parameters of digital signatures) and liveness under partial synchrony.

REFERENCES

[1] Vitalik Buterin. 2013. Bitcoin Network Shaken by Blockchain Fork. *Bitcoin Magazine* (March 13 2013). https://bitcoinmagazine.com/technical/bitcoin-network-shaken-by-blockchain-fork-1363144448.

[2] Vishal Chawla. 2022. Harmony's $100 million hacker took control of its multi-signature wallet, analysts say. *The Block* (June 24 2022). https://www.theblock.co/news/regulation/2022-06-24-harmonys-100-million-hacker-took-control-of-its-multi-signature-wallet-analysts-say-154029.

[3] Tim Copeland. 2023. Multichain ceases operations as CEO's sister is detained in China while holding $220 million of funds. *The Block* (July 14 2023). https://www.theblock.co/news/ecosystems/2023-07-14-multichain-ceases-operations-as-ceos-sister-is-detained-in-china-while-holding-220-million-of-funds-239589.

[4] Tom Dickman. retrieved 2026. PoW 51% Attack Cost. https://www.crypto51.app/.

[5] Stellar Development Foundation. Retrieved October 2026. Tier 1 Organizations. https://developers.stellar.org/docs/validators/tier-1-orgs.

[6] Christopher Goes. 2020. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918 [cs.DC] https://arxiv.org/abs/2006.15918

[7] Aggelos Kiayias, Andrew Miller, and Dionysis Zindros. 2020. Non-interactive Proofs of Proof-of-Work. In *Financial Cryptography and Data Security*, Joseph Bonneau and Nadia Heninger (Eds.). Springer International Publishing, 505–522.

[8] Marta Lokhava, Giuliano Losa, David Mazières, Graydon Hoare, Nicolas Barry, Eli Gafni, Rafał Malinowsky, and Jed McCaleb. 2019. Fast and Secure Global Payments with Stellar. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. Huntsville, Ontario.

[9] David Mazières. 2015. The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus. (April 2015). https://www.stellar.org/papers/stellar-consensus-protocol.pdf

A PROOFS

Proof of Theorem 5.1. Suppose that $I \not\subseteq S$ and that no member of $I \setminus S$ is blocked by $I \cap S$. Then each member of $I \setminus S$ has a slice disjoint from $I \cap S$, whose intersection with I is included in $I \setminus S$, so $I \setminus S$ is a non-empty quorum in $\Sigma \restriction I$. Moreover, each member of $Q \cap I$ has a slice included in Q, so $Q \cap I$ is a non-empty quorum in $\Sigma \restriction I$. Since I has quorum intersection despite its complement, these two quorums intersect, which contradicts $Q \subseteq S$. □

Proof of Lemma 5.6. The claim holds for $c = g_C$ by initialization. Suppose that $c \neq g_C$.

Every blocking set of a member of I intersects I, since I is a quorum. So the first member w of I to accept an import statement does not accept it because a blocking set has: w belongs to a quorum that unanimously voted for the statement, and in particular w voted for it. For import C $\mathsf{parent}_B(c)$ c, this means that c is valid at w, which proves the first part. It also follows that members of I never accept two conflicting statements: the quorums through which the first members of I to accept them did so would each intersect I in a quorum of $\Sigma_B \restriction I$, and since I is intact, these intersect in a member of I, which would have voted for both.

For the second part, consider an import statement that a member u of I confirmed by time t through a quorum Q that contains u. The members of $Q \cap I$ accepted the statement by time t. Let A_k be the set of members of I that have accepted the statement by time $t + k\Delta$, so that $Q \cap I \subseteq A_k$. Each member of A_k has broadcast its acceptance by then, so, if $A_k \neq I$, Theorem 5.1 applied to $S = Q \cup A_k$ gives a member of $I \setminus A_k$ that is blocked by A_k and thus accepts the statement by time $t + (k+1)\Delta$, whether or not c is valid at it; by the above, it has not accepted a conflicting statement. Since $A_0 \supseteq Q \cap I$ is non-empty, $A_{|I|-1} = I$, and, since I is a quorum, every member of I confirms the statement by time $t + |I|\Delta$. This applies to the import statements along the parent path in B's view from c back to g_C, excluding g_C. The member of I that imported c confirmed all these statements by time t. Since g_C is imported initially, every member of I imports c by time $t + |I|\Delta$. □

Proof of Theorem 5.7. A well-behaved validator v commits only headers that its consensus layer commits and that are admissible at v, hence conditionally B-valid, and it externalizes only headers that are B-valid at v. This gives Validity, and Consistency follows from Consistency of consensus.

For the other properties, consider a B-intact set I. Since conditional B-validity depends only on the header and its ancestors, and can be checked within Δ_{val}, Lemma 5.6 implies that admissibility is timely in I, with $D = |I|\Delta + \Delta_{val}$. We first show that if the consensus layer of a member of I commits a header b, then every member v of I eventually commits b. By Validity of consensus, b is admissible at a member of I, hence eventually at v, and, by Totality of consensus, the consensus layer of v eventually commits b. Then v externalizes b once b is B-valid at v, that is, once the foreign headers of b and of its ancestors are valid at v. Totality follows, since a member of I externalizes only headers that its consensus layer committed.

For Liveness, suppose that foreign-header validity is total in I, and consider a member v of I and a foreign header c of a header that v commits, or of one of its ancestors. Since c is imported at v, Lemma 5.6 implies that c is valid at a member of I. Since foreign-header validity is total in I, c eventually becomes valid at v. So v eventually externalizes every header that it commits. Initially, g_B is admissible at every member of I, and each light client has imported its foreign genesis header. The block manager can extend any admissible parent using these or later imported headers, since get_txset eventually returns a valid transaction set. By Liveness of consensus, the consensus layers of members of I commit infinitely many headers, each proposed by a member of I and thus

with some parent b_P, foreign headers H, and a transaction set returned by get_txset(b_P, H) at that member. By the above, every member of I commits, and thus externalizes, each of them.

For Foreign liveness, take b, c_b, and c as in the property. By the above, every member of I eventually commits b. Thus every member eventually imports a common C-header a descending from c_b: take $a = c_b$ when $c_b \neq \bot$, and $a = g_C$ otherwise. By Lemma 5.6 and the no-conflict hypothesis, a is compatible with c in B's view. If c is an ancestor of a, every member's latest imported C-header already descends from c. Otherwise, consider a strict descendant d of a that is an ancestor of c, both in B's view. Since c becomes valid at a member of I and foreign-header validity is total in I, c eventually becomes valid at every member, and so does d (Section 5.4.2). The C-header of a statement that conflicts with IMPORT C $\mathsf{parent}_B(d)$ d descends from c_b and is incompatible with c, both in B's view, so it is never valid at a member of I, and no member of I votes for that statement or, by the proof of Lemma 5.6, accepts it. So every member of I eventually votes for IMPORT C $\mathsf{parent}_B(d)$ d, and, since I is a quorum, eventually confirms it. Since a is imported, c is then eventually imported at every member of I. In either case, the latest imported C-header of each member of I descends from c in B's view, and so does the C-header of every header that it proposes. By Liveness of consensus, members of I commit infinitely many headers proposed by members of I, hence one proposed after that, and, as for Liveness, every member of I externalizes it. □